TeamFlow — How the Backend Is Connected to the Frontend

Project: Team Task Management System | **Frontend:** Angular 20 (`task-flow`) | **Backend:** Spring Boot microservices (`task-management-backend`)

1. System Overview

Component	Technology	Port	Role
auth-service	Spring Boot, PostgreSQL (`auth_db`)	8081	Signup / login, issues JWT tokens
core-service	Spring Boot, PostgreSQL (`core_db`)	8082	Projects, tasks, members, activity
task-flow	Angular (signals, HttpInterceptor)	4200	Web UI, calls both services via HTTP

Request flow

```
Browser (Angular, :4200)
   |  1. POST /api/auth/login            ──►  auth-service (:8081)  → returns JWT
   |  2. Angular stores token in localStorage
   |  3. Every /api/* request — authInterceptor adds  Authorization: Bearer <token>
   ▼
core-service (:8082) — JwtAuthFilter validates the JWT ─► Controller ─► Service ─► Repository
```

2. Step-by-step files used to make the connection

Step 1 — Define the backend URLs in the Angular environment

`task-flow/src/environments/environment.ts` — single place where all API base URLs live.

```ts
export const environment = {
  production: false,
  authApiUrl: 'http://localhost:8081/api/auth',
  coreApiUrl: 'http://localhost:8082/api',
};
```

Step 2 — Configure the HttpClient + interceptor in Angular

task-flow/src/app/app.config.ts — registers the HTTP client and attaches `authInterceptor` globally.

```
providers: [
  provideRouter(routes),
  provideHttpClient(withInterceptors([authInterceptor])),
],
```

Step 3 — Attach the JWT to every outgoing request

task-flow/src/app/Core/interceptors/auth.interceptor.ts — clones the request and adds the `Authorization: Bearer` header for all core-service calls.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  if (req.url.startsWith(environment.coreApiUrl)) {
    const token = authService.token();
    if (token) {
      return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
    }
  }
  return next(req);
};
```

Step 4 — Login / signup and store the session (frontend)

task-flow/src/app/Core/services/auth.service.ts — POSTs to the auth-service and saves the returned JWT in `localStorage` (keys `token` and `user`), exposed through Angular signals.

- `login()` → `POST {authApiUrl}/login` with `{ email, password }`
- `signup()` → `POST {authApiUrl}/signup` with `{ fullName, email, password }`
- `saveSession()` stores token + user; `logout()` clears them and redirects to `/login`

task-flow/src/app/Core/guards/auth.guard.ts — blocks protected routes when there is no token, redirecting to `/login`. Wired in task-flow/src/app/app.routes.ts.

Step 5 — Backend auth endpoints + JWT generation

task-management-backend/auth-service/.../security/SecurityConfig.java

```
.sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
  .requestMatchers("/api/auth/**").permitAll()   // login/signup are public
  .anyRequest().authenticated())
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

`task-management-backend/auth-service/.../service/AuthService.java` — hashes passwords with BCrypt, validates login, and calls `jwtUtil.generateToken(userId, email)`. Returns `AuthResponse { token, userId, fullName, email }` — exactly the shape the Angular `AuthService` expects.

`task-management-backend/auth-service/.../security/JwtUtil.java` — signs an HS256 token (24 h expiry) with `${JWT_SECRET}` from `application.properties`.

Step 6 — Validate the JWT on the core-service

`task-management-backend/core-service/.../security/JwtAuthFilter.java` — reads the `Authorization: Bearer` header sent by the Angular interceptor, verifies it with the same `${JWT_SECRET}`, and puts the userId into the Spring Security context.

`task-management-backend/core-service/.../security/SecurityConfig.java` — stateless sessions; **every** request to core-service must be authenticated.

> **Key point:** both services must use the same `jwt.secret` environment variable, otherwise core-service rejects tokens issued by auth-service.

Step 7 — Allow the browser to call another port (CORS)

`task-management-backend/auth-service/.../security/CorsConfig.java` and `task-management-backend/core-service/.../security/CorsConfig.java` — both allow the Angular dev server origin:

```
registry.addMapping("/api/**")
  .allowedOrigins("http://localhost:4200")   // Angular port
  .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
  .allowedHeaders("*");
```

Step 8 — Expose the REST API in the backend

Controllers under `task-management-backend/core-service/.../`:

Endpoint	File
`GET/POST /api/projects`, `PUT/DELETE /api/projects/{id}`, `POST/DELETE /api/projects/{id}/members/{memberId}`	`project/ProjectController.java`
`GET/POST /api/tasks`, `PUT/DELETE /api/tasks/{id}`, `PATCH /api/tasks/{id}/status`	`task/TaskController.java`
`GET/POST /api/members`, `PUT/DELETE /api/members/{id}`	`member/MemberController.java`
`GET/POST /api/activity`	`activity/ActivityController.java`

Business logic lives in the matching `*Service.java`, persistence in the `*Repository.java` (Spring Data JPA) and entities like `Task.java`, `Project.java`, `Member.java`, `Activity.java`. Errors are handled by

`exception/GlobalExceptionHandler.java`, which returns `400 { "error": "..." }` for `IllegalArgumentException` (e.g. duplicate email).

Step 9 — Call the API from Angular services (frontend)

Each Angular service in `task-flow/src/app/Core/services/` maps its local model to the backend JSON shape (enum values differ: backend uses `TODO/IN_PROGRESS/REVIEW/DONE`, frontend uses `Todo/In Progress/Review/Done`):

Angular service	Backend endpoint	Notes
`project.service.ts`	`/api/projects`	Optimistic create with rollback on error; maps ACTIVE/COMPLETED/ARCHIVED ↔ Active/Completed/Archived
`task.service.ts`	`/api/tasks`	Optimistic create/update; `mapTaskStatus()` / `mapPriority()` convert enums; due dates sent as `yyyy-MM-dd`
`member.service.ts`	`/api/members`	Groups members by `teamName`; maps ONLINE/OFFLINE ↔ Online/Offline
`activity.service.ts`	`/api/activity`	Optimistic add, replaced by server copy on success

Each service also contains an `effect()` that re-fetches data automatically when the auth token changes (login → load, logout → clear).

3. How to run everything

```
 # 1. PostgreSQL must be running with databases auth_db and core_db

# 2. Set environment variables (PowerShell):
$env:DB_PASSWORD = "your-postgres-password"
$env:JWT_SECRET  = "a-long-secret-key-at-least-32-chars"

# 3. Start the backends:
cd task-management-backend\auth-service ; .\mvnw spring-boot:run   # port 8081
cd task-management-backend\core-service ; .\mvnw spring-boot:run   # port 8082

# 4. Start the frontend:
cd task-flow ; npm install ; npm start    # http://localhost:4200
```

4. Verification checklist

- Signup at `http://localhost:4200/signup` → user saved in `auth_db`, JWT returned.
- Login redirects to dashboard; `localStorage.token` contains the JWT.

- Dashboard/tasks pages show data from `core_db` (Authorization header present in DevTools → Network).
- Creating a task/project persists after page refresh (data comes from PostgreSQL, not local mocks).
- Calling core-service without a token returns `401/403`.